

L2 Normalization, Worked Out by Hand

Rescaling every embedding to unit length — the one preprocessing step that makes dot product, cosine, and Euclidean distance all agree — computed on real vectors, with proof that it preserves direction while discarding magnitude.

What this document does. It takes four vectors, divides each by its own L2 norm, and verifies the results are unit length. It then proves the three payoffs: (1) the dot product of normalised vectors equals their cosine, (2) Euclidean and cosine rankings coincide, and (3) two vectors that differ only in length become *identical*. Numbers from Appendix A; vectors match the cosine, dot-product, and Euclidean PDFs.

1 The operation

To L2-normalise a vector, divide it by its own length so the result points the same way but has norm 1:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|} = \frac{\mathbf{v}}{\sqrt{\sum_i v_i^2}} \quad \Rightarrow \quad \|\hat{\mathbf{v}}\| = 1.$$

The hat $\hat{\mathbf{v}}$ denotes the unit vector. Geometrically, every vector is projected onto the surface of the unit sphere; only its *direction* survives.

Intuition

Normalisation is a great equaliser. Before it, a long vector and a short vector that mean the same thing look different to any length-sensitive metric. After it, they are the same point on the unit sphere. You are deciding, once and for all, that for this data *magnitude carries no meaning* — only direction does. For text embeddings that is almost always the right call, which is why most embedding APIs return unit vectors by default.

2 Worked by hand

Norms first: $\|\mathbf{q}\| = 3$, $\|\mathbf{a}\| = \sqrt{5} = 2.2361$, $\|\mathbf{b}\| = \sqrt{10} = 3.1623$, $\|\mathbf{c}\| = 6$. Divide each component by the norm:

$\mathbf{v}$	$\mathbf{v}$	$\ \mathbf{v}\ $	$\hat{\mathbf{v}} = \mathbf{v}/\ \mathbf{v}\ $
$\mathbf{q}$	(2, 1, 2)	3	(0.6667, 0.3333, 0.6667)
$\mathbf{a}$	(1, 0, 2)	2.2361	(0.4472, 0, 0.8944)
$\mathbf{b}$	(0, 3, 1)	3.1623	(0, 0.9487, 0.3162)
$\mathbf{c} = 2\mathbf{q}$	(4, 2, 4)	6	(0.6667, 0.3333, 0.6667)

Check $\hat{\mathbf{a}}$: $0.4472^2 + 0^2 + 0.8944^2 = 0.2 + 0 + 0.8 = 1.0$, so $\|\hat{\mathbf{a}}\| = 1.0$. ✓

Mini-example · length differences vanish

$\mathbf{q} = (2, 1, 2)$ and $\mathbf{c} = (4, 2, 4)$ are different vectors — $\mathbf{c}$ is twice as long. But

$$\hat{\mathbf{c}} = \frac{(4, 2, 4)}{6} = (0.6667, 0.3333, 0.6667) = \hat{\mathbf{q}}.$$

After normalisation they are *the same vector*. The factor of 2 that inflated $\mathbf{c}$'s dot product to 18 (PDF #6) and pushed its Euclidean distance to 3 (PDF #7) is simply gone. Normalisation is what made cosine's verdict ($\mathbf{q} \equiv \mathbf{c}$) the universal one.

3 The three payoffs, proved

(1) **Dot product becomes cosine.** For unit vectors $\|\hat{\mathbf{u}}\| = \|\hat{\mathbf{v}}\| = 1$, so

$$\hat{\mathbf{u}} \cdot \hat{\mathbf{v}} = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \cos(\mathbf{u}, \mathbf{v}).$$

The expensive cosine (a dot plus two norms plus a division) collapses to a single dot product. Check: $\hat{\mathbf{q}} \cdot \hat{\mathbf{a}} = 0.6667 \cdot 0.4472 + 0 + 0.6667 \cdot 0.8944 = 0.2981 + 0.5963 = 0.8944$, the cosine from PDF #5.✓

(2) **Euclidean and cosine agree.** From PDF #7, $\|\hat{\mathbf{u}} - \hat{\mathbf{v}}\|^2 = 2 - 2\cos\theta$, a decreasing function of cosine — so nearest by L2 = largest cosine. All three metrics rank identically.

(3) **Scale invariance is now built in.** Since $\widehat{\alpha\mathbf{v}} = \alpha\mathbf{v}/\|\alpha\mathbf{v}\| = \alpha\mathbf{v}/(\alpha\|\mathbf{v}\|) = \hat{\mathbf{v}}$ for any $\alpha > 0$, no positive rescaling of an input can change the stored vector. Length-based artefacts (passage length, token count) can no longer leak into retrieval scores.

Intuition

The practical upshot for a vector DB: **normalise once at index time**, store unit vectors, and run plain dot-product search forever after. You pay the square root and division a single time per vector instead of on every query, and you get cosine-quality ranking at dot-product speed. This is the standard production setup.

4 One caveat

Normalisation *destroys* magnitude information — by design. If your embeddings deliberately encode something in their length (a confidence, a count, a MIPS model trained that way), normalising throws it away and you should not do it. The question is always the same: *does length mean anything here?* For directional semantic embeddings, no — normalise. For magnitude-bearing vectors, no — keep the length.

Equation cheat-sheet

$$\begin{aligned} \hat{\mathbf{v}} &= \frac{\mathbf{v}}{\|\mathbf{v}\|}, \quad \|\hat{\mathbf{v}}\| = 1, \quad \widehat{\alpha\mathbf{v}} = \hat{\mathbf{v}} \ (\alpha > 0) \\ \hat{\mathbf{u}} \cdot \hat{\mathbf{v}} &= \cos(\mathbf{u}, \mathbf{v}), \quad \|\hat{\mathbf{u}} - \hat{\mathbf{v}}\|^2 = 2 - 2\hat{\mathbf{u}} \cdot \hat{\mathbf{v}} \end{aligned}$$

Appendix A · Reference implementation

```
import numpy as np

def l2_normalize(v):
    v = np.asarray(v, float)
    return v / np.linalg.norm(v)

l2_normalize([2,1,2])    # [0.6667 0.3333 0.6667]
```

```
l2_normalize([4,2,4])    # [0.6667 0.3333 0.6667] == normalize([2,1,2])
np.linalg.norm(l2_normalize([0,3,1]))    # 1.0
```

Internal learning material. Numbers reproducible from the appendix code. v1.0